

FairMind: Causal Fairness Analysis

Report di Panoramica del Progetto

1. Descrizione Generale del Progetto

FairMind è un'implementazione operativa dei concetti di causal fairness presentati nel paper "Causal Fairness Analysis" di Drago Pleško e Elias Bareinboim (FNT in Machine Learning, 2024). La libreria implementa un pipeline "interpretativo" che segue le idee di "LLM as Data Scientist" per utilizzare modelli linguistici (LLM) nell'interpretazione delle metriche di causal fairness calcolate.

Il progetto nasce come tesi e permette di analizzare se un attributo sensibile (es. sesso, razza) ha un effetto causale discriminatorio su un outcome (es. ammissione, reddito), distinguendo tra effetto diretto, indiretto (via mediatori) e spurio (via confondenti).

2. Struttura del Repository

Directory/File	Descrizione
src/	Codice sorgente principale della libreria
src/effects.py	Stima degli effetti causali e decomposizioni (TV, TE, NDE, NIE, SE)
src/model.py	Fitting di Bayesian Network discreti (pgmpy)
src/graph.py	Costruzione del modello SFM (Standard Fairness Model)
src/llm.py	Integrazione con LLM (OpenAI) per report automatici
src/preprocess.py	Preprocessing dei dataset (es. binning)
src/visualisation/	Utilità di visualizzazione (grafi, diagrammi Sankey)
tests/	Suite di test con pytest
data/datasets/	Dataset grezzi (adult, credit_limit, etc.)
data/processed/	Dataset processati (16 file: COMPAS, German Credit, etc.)
experiments/	Report LaTeX degli esperimenti (Adult, Student Mat)
notebooks/	Notebook Jupyter esplorativi e dimostrativi
prompts/	Prompt di sistema per LLM (fairmind_prompts, prompts_onlygpt)
ui/	Applicazione Streamlit interattiva
pyproject.toml	Configurazione del progetto Python (dipendenze, tooling)

3. Tecnologie e Dipendenze Principali

Il progetto utilizza Python ≥ 3.14 e il package manager uv per la gestione delle dipendenze. Ecco le librerie principali:

- pgmpy — modellazione e inferenza con Bayesian Network discreti
- networkx — definizione e manipolazione di grafi causali (SFM)
- numpy, pandas, scipy — elaborazione dati numerici
- plotly, matplotlib, daft-pgm — visualizzazioni (Sankey, grafi PGM)
- openai — integrazione con LLM (gpt-5-mini) per report automatici
- reportlab — generazione di report PDF
- streamlit — interfaccia utente interattiva
- loguru — logging
- kaleido — export di visualizzazioni statiche
- python-dotenv — gestione variabili d'ambiente (API key)
- ruff — linting e formattazione
- pytest + pytest-cov — test con copertura

4. Modello Causale: Standard Fairness Model (SFM)

Il cuore del progetto è il concetto di Standard Fairness Model (SFM), un grafo diretto aciclico (DAG) in cui ogni nodo viene annotato con un tipo:

- **Sensitive (X)** — attributo sensibile protetto (es. sesso, razza)
- **Outcome (Y)** — variabile di risultato (es. ammissione, reddito)
- **Mediator (W)** — variabili mediatrici sul percorso causale da X a Y
- **Confounder (Z)** — variabili confondenti che influenzano sia X che Y
- **Latent (U)** — variabili latenti non osservate

La funzione **build_sfm()** in `src/graph.py` costruisce il grafo con tutti gli archi causali standard: confondenti $\rightarrow$ X, confondenti $\rightarrow$ Y, X $\rightarrow$ mediatori, mediatori $\rightarrow$ Y, X $\rightarrow$ Y, confondenti $\rightarrow$ mediatori. È possibile specificare ordinamenti topologici per mediatori e confondenti, necessari per decomposizioni avanzate.

5. Metriche di Causal Fairness Implementate

Il file **src/effects.py** implementa le seguenti metriche della famiglia Total Variation (TV), basate sul framework di Plečko & Bareinboim:

- **Total Variation (TV)**: Differenza osservazionale $P(Y | X=x1) - P(Y | X=x0)$. Misura la disparità complessiva.
- **Total Effect (TE)**: Effetto causale totale: $P(Y@do(X=x1)) - P(Y@do(X=x0))$. Quantifica l'impatto causale.

- **Natural Direct Effect (NDE)**: Effetto diretto di X su Y mantenendo i mediatori al livello controfattuale di x_0 .
- **Natural Indirect Effect (NIE)**: Effetto indiretto mediato: cambia i mediatori al livello di x_1 ma tiene X a x_0 .
- **Spurious Effect (SE)**: Effetto spurio: differenza tra $P(Y | X=x)$ e $P(Y@do(X=x))$, dovuto ai confondenti.
- **Set-Specific Indirect Effect**: Decomposizione dell'effetto indiretto per singolo mediatore (Theorem 6.6).
- **Set-Specific Spurious Effect**: Decomposizione dell'effetto spurio per singolo confondente (Theorem 6.2).

È inoltre implementata una versione utilitaria (utility-weighted effect) che tratta gli stati dell'outcome come valori numerici tramite una funzione di utilità $T(Y)$, e una versione categorica che calcola matrici di effetti per tutte le coppie di stati.

6. Architettura del Codice Sorgente

- **src/graph.py** (107 righe): Contiene *build_sfm()* per costruire il grafo SFM e *filter_nodes_by_type()* per filtrare nodi per attributo (tipo/categoria).
- **src/model.py** (41 righe): Contiene *fit_discrete_bayesian_model()* che prende un SFM, dati e un estimatore pgmpy (MLE, BayesianEstimator con BDeu/K2/Dirichlet) e restituisce un DiscreteBayesianNetwork.
- **src/effects.py** (1422 righe): Il cuore della libreria. Implementa tutte le metriche, le decomposizioni, la classe EffectResult per matrici di effetti, e le funzioni di report (compute_fairness_report, compute_categorical_fairness_report).
- **src/llm.py** (225 righe): Prepara payload JSON per l'LLM, carica i prompt da file, invia richieste a OpenAI (modello gpt-5-mini con reasoning ad alto sforzo) e restituisce report in formato TEXT + LATEX.
- **src/preprocess.py** (21 righe): Preprocessing del dataset Adult con binning di hours-per-week.
- **src/visualisation/graph.py** (89 righe): Visualizzazione SFM con daft-pgm, con layout automatico per nodi sensibili, outcome, mediatori, confondenti e variabili latenti.
- **src/visualisation/sankey.py** (532 righe): Diagrammi Sankey interattivi con Plotly per decomposizioni di effetti: $TV \rightarrow TE + SE$, $TE \rightarrow DE + IE$, con decomposizioni per mediatore/confondente.

7. Applicazione Streamlit (Interfaccia Utente)

Il file **ui/app.py** (1200 righe) implementa un'applicazione web Streamlit completa per l'analisi interattiva di causal fairness. Funzionalità principali:

- Caricamento dataset (CSV, TSV, Excel, JSON) con pulizia automatica
- Discretizzazione interattiva di variabili numeriche (equal-width, quantile)

- Selezione di X (sensibile), Y (outcome), W (mediatori), Z (confondenti)
- Supporto per Y continuo con analisi di soglia (threshold analysis)
- Visualizzazione del grafo SFM con daft-pgm
- Calcolo di effetti generali e pairwise (matrici per tutte le coppie di stati X)
- Decomposizioni per mediatore/confondente
- Analisi stepwise per X ordinato con curva TE cumulativa
- Download dei risultati in JSON (payload per LLM)
- Generazione di report LLM (TEXT + LaTeX) tramite OpenAI

8. Integrazione con LLM

Due modalità di integrazione con LLM (OpenAI):

- **Con risultati precalcolati (fairmind_prompts.txt):** Il sistema calcola le metriche localmente, le serializza in JSON, e le invia all'LLM che produce un report interpretativo in formato TEXT e LATEX.
- **Solo dataset (prompts_onlygpt.txt):** Il dataset viene inviato all'LLM che deve calcolare autonomamente la decomposizione della fairness seguendo la teoria di Plečko & Bareinboim.

9. Dataset Inclusi

- Adult (UCI) — predizione reddito >50K, attributo sensibile: sesso/razza
- COMPAS — recidiva, attributo sensibile: razza
- German Credit — affidabilità creditizia
- Berkeley — ammissioni università, attributo sensibile: sesso
- Student Mat / Student Por — performance scolastica (matematica/portoghese)
- Bank Marketing — sottoscrizione deposito
- Dutch Census 2001 — censimento olandese
- Law Bar Pass — esame avvocatura
- Diabetes 130 — readmissioni diabetici
- UCI Credit Card — default pagamenti
- Census Income KDD — reddito censimento
- Crime Data — criminalità
- StudentInfo OULAD — educazione online
- Credit Limit — limite di credito

10. Test

Il progetto include una suite di test con pytest (4 file):

- **test_graph.py** (253 righe) — test per `build_sfm` e `visualize_sfm`
- **test_model.py** (304 righe) — test per `fit_discrete_bayesian_model` con vari estimator
- **test_tv_decomposition.py** (162 righe) — verifica identità di decomposizione $TV = TE + (SE(x1) - SE(x0))$
- **test_utilitarian_effects.py** (155 righe) — test per utility-weighted effect

La configurazione pytest include copertura (`cov=src`) con report term-missing.

11. Strumenti di Sviluppo

- **uv** — gestione dipendenze e ambienti virtuali
- **ruff** — linter e formatter (line-length 88, target Python 3.14)
- **pytest** — test runner con `pytest-cov` per copertura

12. Riferimenti

Il progetto si basa sul framework teorico di:

D. Pleško and E. Bareinboim, "Causal Fairness Analysis: A Causal Toolkit for Fair Machine Learning," FNT in Machine Learning, vol. 17, no. 3, pp. 304–589, 2024.

Contatti del team di sviluppo: alessia.berarducci@supsi.ch, eric.rossetto@supsi.ch, alessandro.antonucci@supsi.ch, marco.zaffalon@supsi.ch

13. Riepilogo

FairMind è un progetto completo e ben strutturato che implementa un framework avanzato di causal fairness analysis. Copre l'intero pipeline: definizione del modello causale (SFM), fitting di Bayesian Network, calcolo di molteplici metriche di fairness (TV, TE, NDE, NIE, SE) con decomposizioni per mediatore/confondente, visualizzazioni interattive (Sankey, grafi PGM), un'interfaccia Streamlit user-friendly, e generazione automatica di report interpretativi tramite LLM. Il progetto è corredato da numerosi dataset reali, notebook esplicativi, esperimenti con report LaTeX e una suite di test.